
Amazon Athena – Complete Overview

What is Amazon Athena?

Amazon Athena is a **serverless interactive query service** that lets you analyze data stored in **Amazon S3** using **SQL**. You do not need to manage any infrastructure, and you only pay for the queries you run.

Key Features

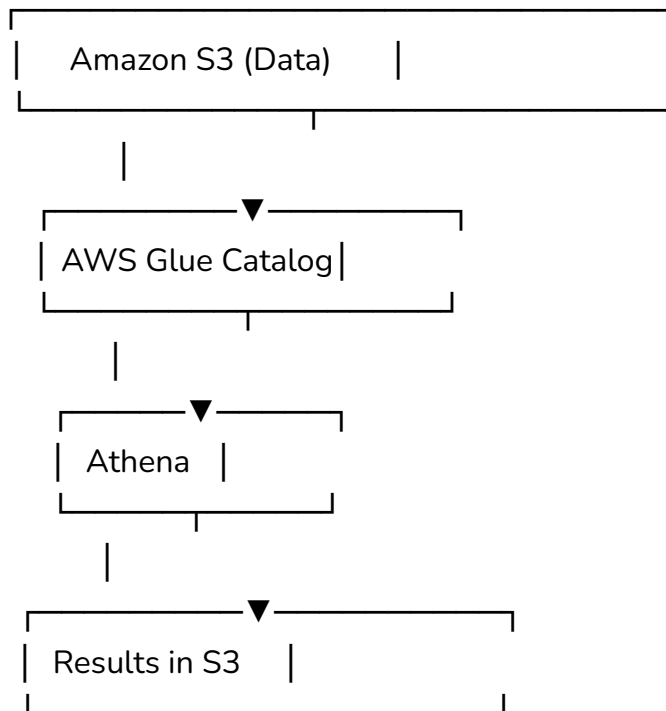
Feature	Description
Serverless	No need to provision or manage infrastructure
Standard SQL Support	Uses ANSI SQL for querying
Pay-per-Query Pricing	Pay only for data scanned by your query
Integrates with Glue	Use AWS Glue Data Catalog for table/schema metadata
Supports Various Formats	CSV, JSON, ORC, Avro, and Parquet
Partitioning Support	Partition your data for performance and cost savings
Highly Available	Built on Presto and scales automatically
Secure	Supports IAM, S3 bucket policies, encryption, and VPC endpoints

Use Cases

- Analyzing **S3 log data** (CloudTrail, ELB logs)

- **Ad hoc querying** for data analysts or developers
 - Running queries over **ETL pipeline output** from AWS Glue
 - Integrating with **Amazon QuickSight** for BI dashboards
 - Performing **data lake analytics** without a data warehouse
-

How It Works



Athena queries structured/unstructured data in S3 using the schema defined in the **AWS Glue Data Catalog**. Results are stored in a query result location in S3.

Pricing

- **\$5 per TB scanned**
 - Optimize cost by:
 - Compressing data
 - Using **columnar formats** (Parquet, ORC)
 - Using **partitions** (e.g., `year=2025/month=07`)
-

Security

- IAM for query permissions
 - S3 bucket policies to limit access
 - Data encryption (in transit & at rest)
 - Integration with **Lake Formation** for fine-grained access control
-

Athena + Glue + S3 Lab Summary

1. Store raw data in S3 (e.g., CSV or Parquet)
2. Create a **Glue Crawler** to catalog data
3. Use **Athena** to run queries on the cataloged tables
4. Store query results in S3

Example Athena SQL Queries

Get top 3 highest-paid employees:

```
SELECT emp_name, salary
FROM employee_data
ORDER BY salary DESC
LIMIT 3;
```

Analyze regional sales:

```
SELECT region, SUM(amount) AS total_sales
FROM sales_data
GROUP BY region;
```

Tips

Optimization Tip	Benefit
Use Parquet format	Faster query & lower cost
Partition large datasets	Scan less data = cheaper
Store compressed files	Lower storage + scan costs
Limit SELECT * usage	Scan only needed columns
